

构想一：波形分解法

核心理念

混合波形 → 短时分解 → 每帧求解各乐器分量 → 时序约束 + 先验知识筛选 → 完整分离

这是你"洗澡时想到"的核心思路，也是当前声源分离领域最主流的模式。

1. 数学模型

1.1 问题形式化

给定混合信号 $y \in \mathbb{R}^T$ (T 为样本点数)，以及 C 种乐器：

$$y(t) = \sum_{i=1}^C x_i(t) \quad \text{for all } t$$

其中 x_i 是第 i 种乐器的纯净信号。

目标：从 y 中估计每个 x_i 。

1.2 你的三步框架 vs. 实际实现

步骤	你的描述	技术实现
① 短时分解	将音频切为极短时间微元	对 y 做 STFT，得到频谱矩阵 $Y \in \mathbb{C}^{F \times N}$ (F = 频率 bins, N = 帧数)
② 每帧求解各乐器分量	对每帧 $Y[:, n]$ 拆分出各乐器	神经网络预测频谱掩码 $M_i \in [0, 1]^{F \times N}$
③ 结果筛选	时序约束 + 乐理校验	网络结构 + Loss 函数 + 后处理
输出	各乐器独立波形	频谱掩码后做 iSTFT → 时域波形

2. 具体方案

2.1 方案 A：频谱掩码 (Spectrogram Masking) —— 推荐作为第一步

这是最成熟、最容易上手的方法。

流程：

混合音频 $y(t)$
↓
STFT → 混合频谱 Y (复数)
↓
└─ 振幅谱 $|Y|$ + 相位谱 $\angle Y$

```

|
▼
神经网络 (U-Net / Hybrid Transformer)
|
▼
预测 C 个掩码 M_1, M_2, ..., M_C
| (每个掩码形状 = |Y| 的形状, 值在 0~1)
|
▼
|X_i| = M_i ⊙ |Y|          (逐元素乘, 提取乐器 i 的振幅)
X_i = |X_i| ⊙ exp(j·∠Y)   (使用混合的相位—近似, 但效果不错)
|
▼
iSTFT → x_i(t)           (还原为时域波形)

```

网络结构 (从简单到复杂) :

Level 1: 线性层 + ReLU + 线性层 + Sigmoid
(baseline, 感受一下“能工作但效果差”是什么体验)

Level 2: 3 层卷积 U-Net
(真正的分离效果, 适合你第一个认真实现的模型)

Level 3: 加入 Transformer 层 + 多尺度处理
(接近 Demucs 水平, 适合后续迭代优化)

对应的简单 PyTorch 骨架:

```

import torch
import torch.nn as nn

class SimpleUNet(nn.Module):
    """最简单的 U-Net 用于声源分离。输入: (batch, 1, F, T) 频谱"""

    def __init__(self, n_freq=1025, n_instruments=4):
        super().__init__()
        # Encoder: 逐步缩小
        self.enc1 = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
            nn.Conv2d(16, 16, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2)) # F/2, T/2
        self.enc2 = nn.Sequential(
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
            nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2)) # F/4, T/4
        # Bottleneck
        self.bottleneck = nn.Sequential(
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
            nn.Conv2d(64, 64, 3, padding=1), nn.ReLU())
        # Decoder: 逐步放大 + 跳连
        self.dec2 = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 2, stride=2),

```

```

        nn.ReLU())
    self.dec1 = nn.Sequential(
        nn.ConvTranspose2d(32, 16, 2, stride=2),
        nn.ReLU())
    # 输出每个乐器的掩码
    self.out = nn.Sequential(
        nn.Conv2d(16, n_instruments, 1),
        nn.Sigmoid()) # 掩码值在 0~1

def forward(self, x):
    """x: (B, 1, F, T)"""
    e1 = self.enc1(x)
    e2 = self.enc2(e1)
    b = self.bottleneck(e2)
    d2 = self.dec2(b) + e2 # 跳连
    d1 = self.dec1(d2) + e1 # 跳连
    masks = self.out(d1) # (B, C, F, T)
    return masks

```

注意：上面的代码仅展展示意图。实际需要调整 padding 以匹配维度、处理跳连的尺寸对齐等。

2.2 方案 B：时域直接分离 (Waveform-to-Waveform)

Demucs 的核心做法，直接用 1D 卷积在原始波形上做分离。

```

混合波形 y(t) (T 个样本点)
    |
    ▼
1D 卷积 Encoder → 下采样特征
    |
    ▼
LSTM / Transformer 层 (捕捉长程依赖)
    |
    ▼
1D 卷积 Decoder → 各乐器波形
    |
    ▼
x1(t), x2(t), ..., xc(t)

```

优点：不需要维护相位信息（iSTFT 的相位近似是误差源之一）。**缺点：**训练更慢，需要更多数据，对音频长度更敏感。

2.3 方案 C：混合方法 (Hybrid)

Demucs Hybrid 的做法：时域分支 + 频域分支同时处理，输出融合。

这是当前 SOTA。你可以在实现方案 A 后再升级到这里。

3. 时序连续性在你的框架中如何落地？

你的构想提到了这点，它对应到实际实现中的多个层面：

3.1 网络结构层面（隐式约束）

- **CNN 的局部性**：卷积核同时覆盖时间和频率维度，相邻帧天然共享权重 → 输出平滑
- **跳连 (skip connections)**：decoder 能看到 encoder 的精细结构，有助于保持瞬态和起音 (attack) 的时序精度
- **Transformer**：自注意力机制显式建模帧间关系 ("第 5 帧是第 3 帧的发展")

3.2 Loss 函数层面（显式约束）

```
# 基础 Loss: L1 或 L2 距离
loss_l1 = |predicted_mask - true_mask|.mean()

# 时序平滑 Loss: 相邻帧的差异应尽量小
def temporal_continuity_loss(masks):
    """masks: (B, C, F, N) 其中 N 是帧维"""
    diff = masks[:, :, :, 1:] - masks[:, :, :, :-1]
    return diff.pow(2).mean()

loss = loss_l1 + λ * temporal_continuity_loss(predicted_masks)
```

3.3 后处理层面（可选的显式平滑）

```
from scipy.ndimage import median_filter

# 对预测掩码做中值滤波，去除时序上的孤立噪点
smoothed_mask = median_filter(prediction, size=(1, 1, 1, 5)) # 时间轴上窗口=5帧
```

4. "乐理先验"如何融入？

你原文中的第三个约束——"让模型习得音乐底层逻辑与乐器发声特征"——在现代深度学习中主要通过两种方式实现：

4.1 数据驱动的隐式学习

这是目前的主流做法：

- 用大量多轨音乐数据 (MUSDB18、Slakh2100) 训练模型
- 模型在训练过程中自动学会：不同乐器的频谱模板、常见和弦进行下的乐器配合模式、不同音区乐器能量分布
- **不需要显式编码乐理规则**——模型从数据中统计得到

4.2 显式乐理约束（前沿方向）

你在项目后期可以尝试的实验性做法：

- **音高引导**：在模型中引入音高预测辅助任务 (multi-task learning)，强制模型理解音符
- **和弦约束**：同一和弦内的乐器应该和谐 (harmonic consistency)
- **乐器基频模板**：钢琴的泛音结构是 1:2:3:4... 倍频关系，可以作为结构先验注入网络

```
# 概念示例：基频 (F0) 一致性约束
def harmonic_consistency_loss(spectrum, f0):
    """
```

```
给定预测的乐器频谱和检测到的基频  $F_0$ ，
检查频谱能量是否集中在  $F_0$  的整数倍附近。
"""

# 对每个时间帧，找到  $F_0$  的谐波位置
# 惩罚谐波位置之外的能量
...
```

这不是标准做法——它属于探索方向。第一版不需要这个。

5. 数据需求

阶段	需要什么数据	推荐来源
原型验证	人工合成的混合音频	用 MIDI 文件 + 音源库（如 FluidSynth）合成
正式训练	多轨录音数据集	MUSDB18、Slakh2100
评估	分离后与真实乐器对比	用 SDR/SIR/SAR 打分

人工合成数据（第一版就可以用）

用 MIDI 文件生成你的训练数据：

```
import midi2audio # 或直接使用 fluidsynth
import librosa
import numpy as np

def create_mixture(midi_file, instrument_map):
    """
    从 MIDI 文件生成多乐器混合音频。
    instrument_map: {'piano': 0, 'violin': 41, ...}
                    键 = 乐器名称, 值 = General MIDI 音色编号
    """
    stems = {}
    for name, program in instrument_map.items():
        # 用 FluidSynth 渲染为该乐器的音频
        audio = render_midi_with_instrument(midi_file, program)
        stems[name] = audio

    # 混合
    mixture = sum(stems.values())
    return mixture, stems # 混合 + 各乐器独立轨（作为训练标签）
```

这种方法的好处：**你有 ground truth**（每件乐器的纯净音频），可以直接计算 loss。

6. 初版实现步骤

Step 1：合成数据 + 简单模型（2-4 周）

输入：3 个 MIDI 音符（钢琴、吉他、贝斯各弹一个 C 大三和弦）
输出：能否正确分离出三个乐器的频谱？

用最简单的 2 层全连接网络，确认整个 pipeline 能跑通。

Step 2：用真实音乐跑 U-Net (4-8 周)

输入：MUSDB18 中的一段混合音频（~10 秒）
输出：4 个分离轨（人声、贝斯、鼓、其他）

这时你应该能听到分离效果——虽然不完美，但能听出每个轨道的轮廓。

Step 3：优化与迭代（持续）

- 加入你构想二的思路作为辅助
- 尝试 Hybrid 架构
- 针对特定乐器（如你喜欢的钢琴）优化
- 尝试你的“时序连续性 Loss”和其他自定义约束

7. 一句话总结

构想一的核心：把声源分离建模为“在频谱图上做图像分割 + 时序平滑”的问题，用 U-Net 或类似的架构从混合频谱预测各乐器的掩码。你的时序连续性和乐理先验——分别由网络结构、Loss 函数、训练数据编码实现。